~/majibill/docs/03-backend-database.md

$ cat 03-backend-database.md

MAJIBILL PLATFORM · BACKEND

Backend Engineering & Database Strategy

Module design, tenancy in code, the billing engine, the ledger, payments, sync, API conventions, and how PostgreSQL is configured to carry a county.

```
SELECT set_config('app.tenant_id', $1, true);
-- RLS filters every query to the current tenant
-- money = bigint minor units; ledger is append-only
```

DOC
MB-03

VERSION
1.0 draft

DATE
08 Oct 2026

STATUS
For review

AUDIENCE
Backend, platform and QA engineers

CLASS
Confidential

// Contents

1. Backend structure 3

2. The billing engine 4

3. The ledger 5

4. Readings, validation and sync 6

5. Jobs and workflows 6

6. API conventions 7

7. Database strategy 8

// 1. Backend structure

The core API is a NestJS modular monolith. Each module is a vertical slice with the same internal shape, so humans and agents can navigate any module the same way. Domain logic is pure and unit-testable; side effects live at the edges.

● ● ● module anatomy

```
modules/billing/
  api/            # controllers, DTOs (Zod), OpenAPI decorators
  application/    # use cases: RunBillCycle, ApproveAdjustment, ReissueBill
  domain/         # pure: Tariff, BlockRate, BillCalculator, Money, rounding
  infra/          # repositories (SQL), queue producers, adapters
  events/         # published event types + handlers
  billing.module.ts
  index.ts        # the ONLY file other modules may import
```

Layer	Rule
api	Validate input (Zod), authenticate, authorise, call exactly one use case. No business logic.
application	Orchestrates a transaction, calls domain objects, writes outbox events and audit events.
domain	No I/O, no clock, no randomness. Takes data in, returns decisions. This is where billing correctness lives.
infra	SQL and adapters. Parameterised queries only. Every query runs inside a tenant-scoped transaction.

1.1 Tenancy in code

A request interceptor resolves the tenant (from host, token claim, or API key), stores it in `AsyncLocalStorage`, and every database access goes through a wrapper that opens a transaction and sets the tenant for that transaction only. Code that tries to query without a tenant context throws.

● ● ● typescript

```
// infra/db/with-tenant.ts (illustrative)
export async function withTenant(fn: (tx: Tx) => Promise): Promise {
  const tenantId = tenantContext.require();            // throws if missing
  return pool.transaction(async (tx) => {
    await tx.query("SELECT set_config('app.tenant_id', $1, true)", [tenantId]);
    return fn(tx);                                      // RLS now applies
  });
}
```

● ● ● sql

```
-- one policy pattern for every tenant-owned table
ALTER TABLE bills ENABLE ROW LEVEL SECURITY;
ALTER TABLE bills FORCE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation ON bills
  USING      (tenant_id = current_setting('app.tenant_id', true)::uuid)
  WITH CHECK (tenant_id = current_setting('app.tenant_id', true)::uuid);
-- app role: NOSUPERUSER NOBYPASSRLS; migrations run as a separate owner role
```

RISK Connection pooling trap
Session-level settings leak across clients behind a transaction-mode pooler. Always use `set_config(..., true)` (transaction-local) inside an explicit transaction, as above, never plain `SET`.

// 2. The billing engine

Billing is a pure function of **(readings, tariff version, account state, charge rules)**. It must be deterministic, replayable, and explainable line by line. The same inputs always produce the same bill; a re-run never creates a second bill.

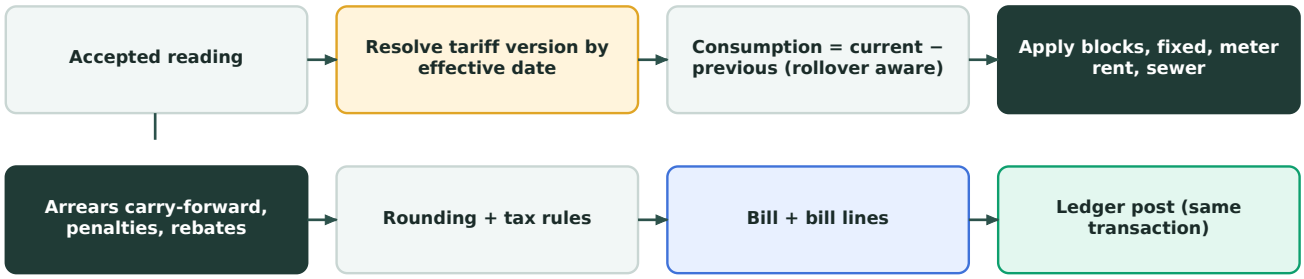


Fig 2.1 Billing pipeline for one account-cycle. Inputs and outputs are persisted so any bill can be explained and re-derived.

2.1 Tariff model

- **Effective-dated versions:** a tariff change creates a new version; bills reference the version used. History is never rewritten.
- **Block (tiered) rates**, fixed monthly charge, meter rent, sewer as a percentage or fixed amount, connection/reconnection fees, penalties on overdue, rebates.
- **Customer classes** (domestic, commercial, institutional, kiosk) map to tariff plans per tenant.
- **Tax and levies** configurable per charge type; rules differ by tenant and over time, so none are hard-coded.

Worked example using the consumption in ZUCAR's deck (120 → 135 m³). **Rates below are illustrative only** and will be replaced with the utility's approved tariff.

Line	Quantity	Rate (KES, illustrative)	Amount (KES)
Block 1: 0–6 m³	6	40.00	240.00
Block 2: above 6 m³	9	60.00	540.00
Fixed monthly service charge	1	150.00	150.00
Meter rent	1	50.00	50.00
Current bill	15 m³		**980.00**
Arrears brought forward			320.00
Amount due			**1,300.00**

2.2 Money rules

- Amounts are stored as `bigint` **minor units** (cents) with a currency code; rates as `numeric(12,4)`. No floating point anywhere.
- One rounding point per line, documented and tested (round half up to cent unless a tenant policy says otherwise).
- Estimates for unread meters use a configurable method (average of last 3 reads, or minimum charge) and are flagged on the bill; the next actual read trues up.
- Meter replacement closes the old meter with a final read and opens the new one with an opening read; consumption never spans two meters.

// 3. The ledger

Every financial effect is a set of immutable ledger entries that balance. Customer balance, arrears ageing, revenue and cash reports are all queries over the ledger. Corrections are new entries that reverse earlier ones, linked by `reverses_entry_id`.

Event	Debit	Credit	Notes
Bill issued	Receivables (customer)	Water revenue, fixed charges, meter rent, sewer	One entry per bill line
Payment received (M-Pesa)	M-Pesa clearing	Receivables (customer)	Allocated to oldest bills first
Unmatched payment	M-Pesa clearing	Suspense	Reclassified when matched
Overpayment	M-Pesa clearing	Customer credit (prepayment)	Auto-applied to next bill
Penalty	Receivables	Penalty income	Rule-driven, reversible
Adjustment (approved)	Adjustments / Receivables	Receivables / Adjustments	Maker–checker required
Write-off	Bad-debt expense	Receivables	Policy threshold + approval
Reversal	(mirror of original)	(mirror of original)	Never edits the original

Ledger invariants (checked nightly and in tests)

- For every journal: sum of debits equals sum of credits.
- Customer balance equals the sum of that account's ledger entries; the cached balance equals the computed balance.
- No update or delete is permitted on ledger tables (grants + trigger).
- Every entry links to its source (bill, payment, adjustment) and to an audit event.

3.1 Payment allocation

1. Resolve account. Apply any unallocated credit already on the account.
2. Allocate to open bills **oldest-first** by default; tenants may choose "charge-type priority".
3. Remaining amount becomes customer credit (prepayment), never silently discarded.
4. Emit `payment.received`, write audit event, enqueue receipt SMS.

3.2 Payment matching

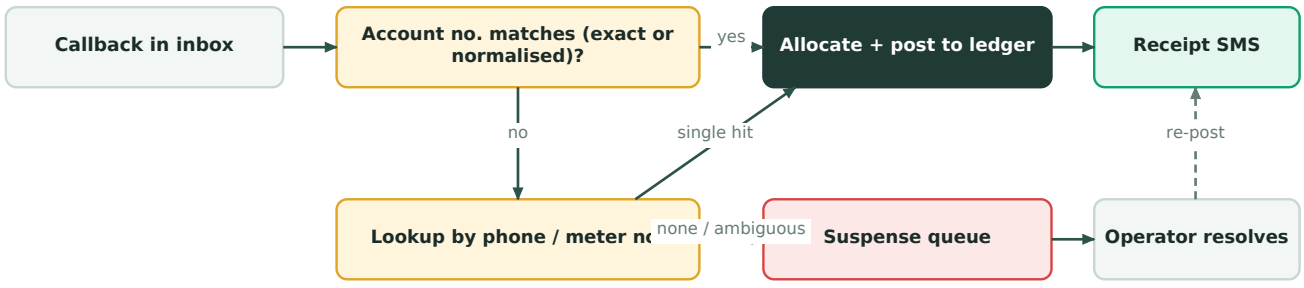


Fig 3.1 Matching ladder. Ambiguous or unknown payments never block the callback and never vanish.

// 4. Readings, validation and sync

Rule	Behaviour
Non-negative	Current below previous is rejected unless flagged as rollover or meter replacement
High / low consumption	Reading above N× the account’s trailing average, or zero for several months, is flagged for supervisor review
Duplicates	Same meter + cycle + value from same device collapses to one reading (client UUID)
Evidence	Photo of meter face required above a configurable consumption jump; GPS tolerance checked against meter location
Locked cycles	Readings after cycle freeze become next cycle’s opening or an approved late correction
Corrections	Never overwrite: create a corrected reading that supersedes the old one; bills re-derive via adjustment

4.1 Mobile sync protocol

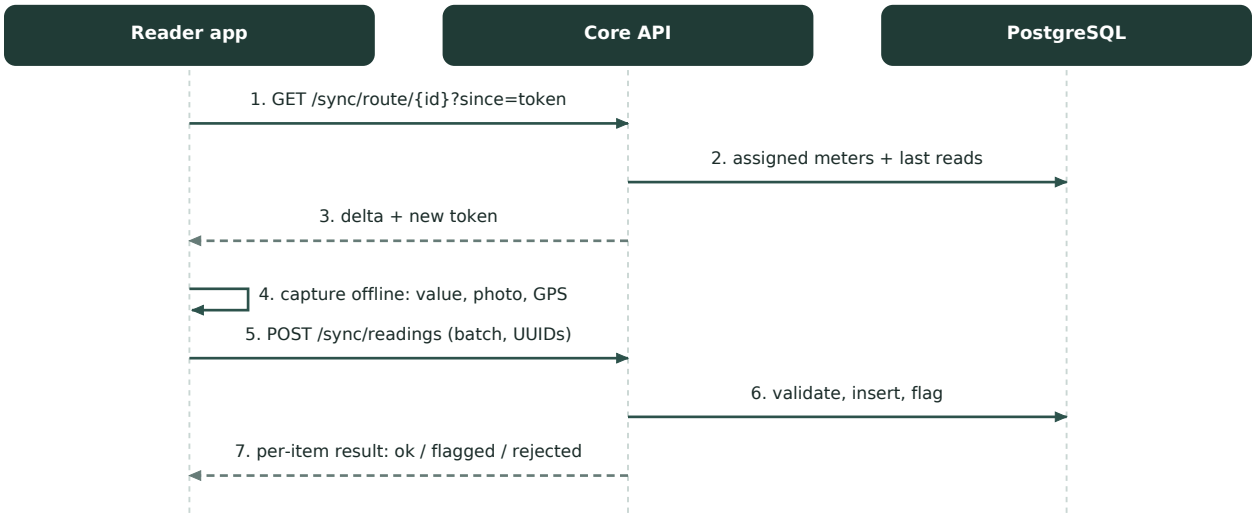


Fig 4.1 Delta download, offline capture, batched upload with per-item results.

- Conflict policy: **server wins on money, field wins on facts.** The device’s reading value is accepted if valid; billing state is always server-side.
- A flagged reading stays in the supervisor queue and does not block the rest of the batch.
- Photo uploads use pre-signed URLs and resume; the reading is accepted before the photo arrives and marked “evidence pending”.

// 5. Jobs and workflows

Queue	Trigger	Idempotency key	Retry
`bill-calc`	Reading accepted / bill run	tenant + account + cycle	5, backoff, then DLQ
`payment-process`	Inbox row	tenant + provider + txn id	10, backoff, then suspense
`notify-sms`	Bill issued, receipt, reminders	notification id	3, then mark failed
`render-pdf`	Statement or bill document requested	document id + version	3
`reconcile-mpesa`	Nightly schedule	tenant + date	Alert on any gap
`penalty-run`	Cycle due date passed	tenant + cycle	Resume
`export`	User request	export id	3
`audit-anchor`	Daily	tenant + date	Alert on failure

// 6. API conventions

Topic	Convention
Style	REST over JSON, OpenAPI 3.1 as the source of truth; clients generated for TypeScript, Dart, Go, Python
Versioning	`/v1/` path prefix; additive changes only inside a version; deprecation headers and a documented window
Auth	Session cookie for web, short-lived JWT + refresh for mobile, scoped API keys for partners; every token carries tenant and role claims
Idempotency	`Idempotency-Key` header required on all POSTs that create money or notifications; stored 24 h+
Pagination	Cursor-based (`?cursor=`, `?limit=`); never offset on large tables
Errors	RFC 9457 `application/problem+json` with stable `code`, `trace_id`, field errors
Rate limits	Per tenant and per token via Redis; `429` + `Retry-After`
Webhooks	HMAC-signed, timestamped, retried with backoff, replay-safe event ids
Time & money	ISO 8601 UTC timestamps; amounts as integer minor units + currency; never floats

```http
POST /v1/payments/manual
Idempotency-Key: 6f2c...        Authorization: Bearer
{ "account_no": "NRB-004417", "amount_minor": 130000, "currency": "KES",
  "method": "cash", "reference": "RCPT-88231", "received_at": "2026-10-08T09:41:00Z" }

201 Created
{ "payment_id": "019...", "allocations": [{ "bill_id": "019...", "amount_minor": 130000 }],
  "balance_minor": 0, "receipt_url": "/v1/receipts/019..." }
```

6.1 Representative endpoints

Area	Endpoints (illustrative)
Customers	`GET/POST /v1/customers`, `GET /v1/accounts/{id}`, `POST /v1/accounts/{id}/close`
Meters & readings	`POST /v1/meters`, `POST /v1/meters/{id}/replace`, `POST /v1/readings`, `GET /v1/readings?flagged=true`
Billing	`POST /v1/cycles`, `POST /v1/cycles/{id}/run`, `POST /v1/cycles/{id}/approve`, `GET /v1/bills/{id}`
Payments	`POST /v1/payments/manual`, `POST /v1/payments/{id}/reverse`, `GET /v1/suspense`, `POST /v1/suspense/{id}/resolve`
Reports	`GET /v1/reports/arrears`, `/collections`, `/consumption`, `/wasreb-kpis`, exports via `POST /v1/exports`
Admin	`/v1/users`, `/v1/roles`, `/v1/tariffs`, `/v1/settings`, `/v1/audit-events`
Platform (console only)	`/platform/v1/tenants`, `/partners`, `/plans`, `/cells`, `/support-sessions`

// 7. Database strategy

Topic	Decision
Engine	PostgreSQL 16 or newer, managed with Multi-AZ in SaaS cells; plain Postgres in sovereign installs
Identifiers	UUIDv7 (time-ordered) primary keys; human-readable `account_no`, `bill_no`, `receipt_no` generated per tenant by sequence
Tenant key	`tenant_id` first in every composite index and every primary access path
Money	`bigint` minor units + `currency char(3)`; rates `numeric(12,4)`
Partitioning	Monthly range partitions on `readings`, `bills`, `ledger_entries`, `payments`, `audit_events` (pg_partman); old partitions detached/compressed per retention
Pooling	PgBouncer (transaction mode) in front; transaction-local tenant setting
Replicas	One read replica for reports and exports; heavy reports never hit the primary
Search	`pg_trgm` for names/phones; normalised phone column (E.164)
Deletes	No hard deletes of financial rows; customer erasure is anonymisation of PII fields with ledger retained
Extensions	pg_partman, pg_trgm, pgcrypto, pgaudit (infra-level audit), pg_stat_statements
Migrations	Plain SQL, reviewed, forward-only, expand/contract; online-safe (CONCURRENTLY, lock timeouts)
Backups	Continuous WAL archiving + PITR, daily snapshots, encrypted cross-region copy, quarterly restore drill

7.1 Performance playbook

- Bulk writes via batched `INSERT ... SELECT` / `COPY` in bill runs; commit per route-chunk, not per account.
- Use advisory locks per `(tenant, cycle)` so two bill runs never overlap.
- Index budget: justify every index with a query; append-only partitions keep vacuum cheap.
- Materialised views for dashboards, refreshed on schedule; arrears ageing computed from ledger snapshots, not on the fly.
- Statement timeouts and `lock_timeout` on every connection; slow-query log feeds a weekly review.

7.2 Testing the backend

Test type	What it proves	Tooling
Golden bill tests	Fixed inputs always produce the same bill lines and totals	Vitest + fixture files reviewed by finance
Property tests	Ledger balances; allocation never exceeds payment; idempotent replays	fast-check
RLS tests	Tenant A cannot read or write tenant B through any module	Integration tests + CI policy-coverage check
Contract tests	OpenAPI matches implementation; clients compile	Schemathesis / Dredd
Sync tests	Offline batches with duplicates, reordering, partial failure	Simulated device harness
Load tests	Bill run, payment burst, report concurrency	k6 against a production-sized cell